



SQL CASE WHEN: The Complete Guide With Examples (2026)

Learn SQL CASE WHEN with practical examples. Master simple CASE, searched CASE, CASE with aggregates, and common patterns every SQL developer needs.

SQL CASE WHEN is the if-then-else of SQL. It lets you add decision-making logic directly inside your queries, transforming, categorizing, and filtering data based on conditions you define.



Quick Navigation

- Simple CASE vs Searched CASE
- How CASE Evaluates: Short-Circuit Logic
- CASE WHEN in SELECT: Categorize Your Data
- CASE WHEN with Aggregates: The Pivot Pattern
- CASE WHEN in WHERE, ORDER BY, and GROUP BY
- Nested CASE and Complex Patterns
- Common Mistakes with CASE WHEN
- Quick Reference: CASE WHEN Cheat Sheet
- FAQ

Simple CASE vs Searched CASE: Know the Difference

SQL has two CASE syntax forms. They look similar but handle different situations. Simple CASE compares one expression against multiple values (like a switch statement). Searched CASE evaluates independent boolean conditions (like an if-else chain).

Simple CASE:

```
SELECT case_id, status,  
       CASE status  
         WHEN 'open'   THEN 'Active Investigation'  
         WHEN 'closed' THEN 'Case Resolved'  
         WHEN 'cold'   THEN 'Inactive'  
         ELSE 'Unknown'  
       END AS status_label  
FROM crime_cases;
```

Simple CASE checks status against each WHEN value using equality (=). Clean and readable for straightforward lookups.

Searched CASE:

```
SELECT suspect_id, name, prior_offenses,  
       CASE  
         WHEN prior_offenses > 5 THEN 'High Risk'  
         WHEN prior_offenses > 2 THEN 'Medium Risk'  
         WHEN prior_offenses > 0 THEN 'Low Risk'  
         ELSE 'Clean Record'
```

```
END AS risk_level
FROM suspects;
```

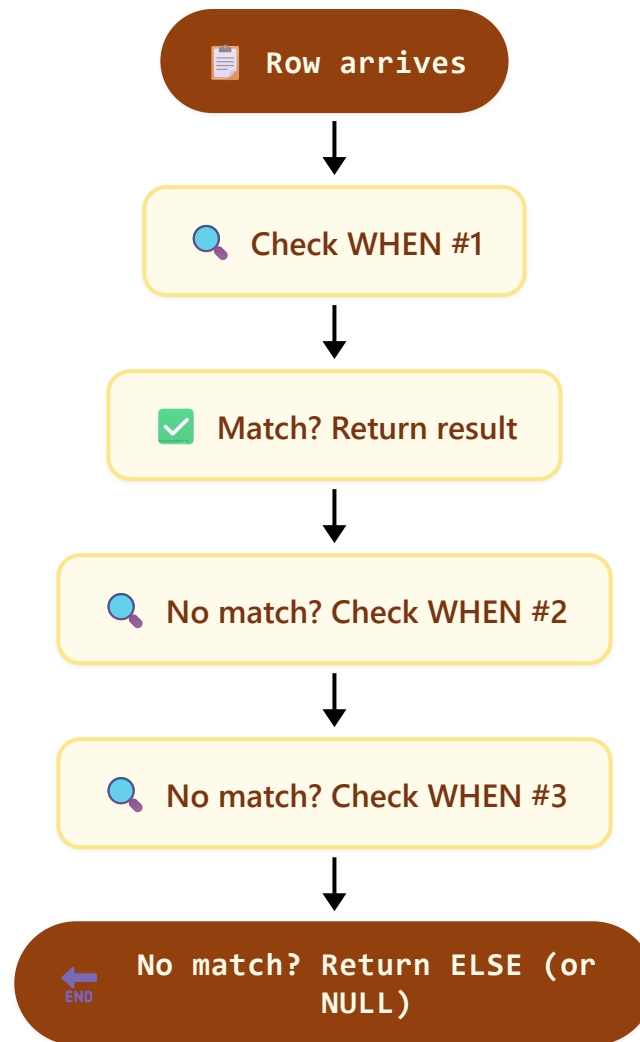
Searched CASE evaluates each WHEN as a standalone condition. Supports ranges, comparisons, LIKE, IN, IS NULL, and anything else you can put in a WHERE clause.

Feature	Simple CASE	Searched CASE
Syntax	CASE expr WHEN val THEN...	CASE WHEN condition THEN...
Comparison type	Equality only (=)	Any comparison (>, <, LIKE, IN, IS NULL)
Multiple columns?	No (single expression)	Yes (any columns)
Range checks?	No	Yes (BETWEEN, >, <)
Best for	Status codes, enum values	Complex business logic
Readability	Cleaner for simple lookups	More explicit conditions

When to use each CASE form

How CASE Evaluates: Short-Circuit Logic

CASE evaluates WHEN conditions top to bottom and returns the result for the first match. Once a condition is true,



CASE evaluates top-to-bottom and stops at the first match

Order matters. Watch this:

```
-- ❌ WRONG: General condition catches everything first
SELECT name, prior_offenses,
       CASE
         WHEN prior_offenses > 0 THEN 'Some Risk'
         WHEN prior_offenses > 5 THEN 'High Risk' -- Never reached!
       END AS risk_level
FROM suspects;

-- ✅ RIGHT: Specific conditions first
SELECT name, prior_offenses,
       CASE
         WHEN prior_offenses > 5 THEN 'High Risk'
         WHEN prior_offenses > 0 THEN 'Some Risk'
         ELSE 'Clean Record'
       END AS risk_level
FROM suspects;
```

In the wrong version, a suspect with 8 prior offenses matches `> 0` first and gets "Some Risk" instead of "High Risk." Always put specific conditions before general ones.

⚠️ Order Matters!

Put your most specific conditions first. `CASE WHEN score > 0 THEN 'some' WHEN score > 100 THEN 'high'` will never reach 'high' because `> 0` catches everything first.

CASE WHEN appears in almost every real SQL query. If you want to practice writing conditional logic hands-on, [SQLNoir's detective cases](#) let you classify evidence and filter suspects using the same patterns.

CASE WHEN in SELECT: Categorize Your Data

The most common use of CASE WHEN is creating new computed columns. Bucketing continuous values into categories, mapping status codes to readable labels, or flagging rows that meet certain criteria.

CASE	CASE	<i>Start the expression</i>
WHEN	WHEN victim_count > 3	<i>First condition to check</i>
THEN	THEN 'Critical'	<i>Value if condition is true</i>
WHEN	WHEN victim_count > 1	<i>Second condition (only reached if first fails)</i>
THEN	THEN 'Major'	<i>Value for second condition</i>
ELSE	ELSE 'Minor'	<i>Default if no conditions match</i>
END	END AS severity	<i>Close expression + name the column</i>

Anatomy of a CASE WHEN expression

Full example:

```
SELECT
  case_id,
  location,
  victim_count,
  CASE
    WHEN victim_count > 3 THEN 'Critical'
    WHEN victim_count > 1 THEN 'Major'
    ELSE 'Minor'
```

```
FROM crime_scenes
ORDER BY victim_count DESC;
```

Each row gets a severity label based on victim_count. Always use AS alias after END to name the computed column.

CASE WHEN with Aggregates: The Pivot Pattern

Put CASE inside aggregate functions like COUNT or SUM to conditionally aggregate data. This is the standard way to pivot rows into columns, and it works in every SQL dialect.

✗ Raw Data (many rows per district)

```
SELECT district, status, COUNT(*)
FROM crime_cases
GROUP BY district, status;
```

- Multiple rows per district
- Hard to compare across statuses
- Needs post-processing for dashboards

✓ Pivoted with CASE (one row per district)

```
SELECT district,
  COUNT(CASE WHEN status = 'open' THEN 1 END) AS open,
  COUNT(CASE WHEN status = 'closed' THEN 1 END) AS closed,
  COUNT(CASE WHEN status = 'cold' THEN 1 END) AS cold
FROM crime_cases
GROUP BY district;
```

- One row per district
- Easy to compare statuses side by side
- Dashboard-ready output

The pivot pattern: turn row values into columns using CASE inside COUNT

Conditional SUM example:

```
SELECT district,  
       SUM(CASE WHEN status = 'closed' THEN evidence_count ELSE 0 END) AS solved_evidence,  
       SUM(CASE WHEN status = 'open' THEN evidence_count ELSE 0 END) AS unsolved_evidence  
FROM crime_cases  
GROUP BY district;
```

SUM with CASE lets you sum different subsets in the same query. Use ELSE 0 with SUM to avoid NULL in the totals.



COUNT vs SUM with CASE

With COUNT, use THEN 1 END (no ELSE needed, COUNT ignores NULLs). With SUM, use ELSE 0 END to avoid NULL results when no rows match.

CASE WHEN in WHERE, ORDER BY, and GROUP BY

CASE is not limited to SELECT. You can use it in ORDER BY for custom sorting, GROUP BY for computed categories, and even WHERE for conditional filtering (though you rarely should).

Custom sort with ORDER BY CASE:

```
-- Show armed suspects first, then by name  
SELECT name, status
```

```
ORDER BY
  CASE WHEN status = 'armed' THEN 0
        WHEN status = 'wanted' THEN 1
        ELSE 2
  END,
  name ASC;
```

CASE in ORDER BY lets you define a custom priority. Armed suspects appear first regardless of alphabetical order.

Group by computed categories:

```
SELECT
  CASE
    WHEN date_opened >= '2025-01-01' THEN 'Recent'
    WHEN date_opened >= '2020-01-01' THEN 'Older'
    ELSE 'Cold Case'
  END AS case_age,
  COUNT(*) AS total
FROM crime_cases
GROUP BY
  CASE
    WHEN date_opened >= '2025-01-01' THEN 'Recent'
    WHEN date_opened >= '2020-01-01' THEN 'Older'
    ELSE 'Cold Case'
  END;
```

The CASE expression must be repeated in both SELECT and GROUP BY (or use a subquery/CTE to avoid repetition).



Skip CASE in WHERE When You Can

You rarely need CASE in WHERE. Instead of `WHERE CASE WHEN type = 'A' THEN 1 ELSE 0 END = 1`, just write `WHERE type = 'A'`. CASE in WHERE is a code smell. Use it only when the filtering logic genuinely depends on other column values.

CASE FILE #004



The Vanishing Diamond ●●○ Intermediate

The diamond vanished from a charity gala. Can you use CASE WHEN to classify guest behavior and identify the thief?

Start Investigation →

Nested CASE and Complex Patterns

You can nest CASE inside CASE, but deep nesting gets hard to read fast. When you find yourself going more than two levels deep, flatten the conditions or use a [CTE](#) with a lookup table instead.

✗ Deeply nested CASE (hard to maintain)

```
CASE WHEN type = 'A'
  THEN CASE WHEN status = 'active'
    THEN CASE WHEN priority > 5
      THEN 'urgent-A'
```

✓ Flat CASE with combined conditions

```
CASE
  WHEN type = 'A' AND status = 'active' AND priority > 5 THEN 'urgent-A'
  WHEN type = 'A' AND status = 'active' THEN 'normal-A'
  WHEN type = 'A' THEN 'inactive-A'
```

```

    END
    ELSE 'inactive-A'
  END
  WHEN type = 'B' THEN ...
END

```

- 3 levels deep
- Hard to read and debug
- Adding a new type requires editing nested structure

```

  WHEN type = 'B' THEN ...
END

```

- Flat structure, easy to scan
- Each outcome on one line
- Adding new cases is straightforward

Flatten nested CASE by combining conditions with AND

CASE with NULL handling:

```

-- CASE for explicit NULL handling
SELECT name, phone,
  CASE
    WHEN phone IS NULL THEN 'No phone on file'
    ELSE phone
  END AS contact_phone
FROM suspects;

-- COALESCE is a shortcut for this exact pattern
SELECT name, COALESCE(phone, 'No phone on file') AS contact_phone
FROM suspects;

```

COALESCE returns the first non-NULL value. When your CASE just replaces NULL with a default, use COALESCE instead.

Three mistakes trip up almost everyone who writes CASE WHEN queries. Here they are with fixes.

Mistake 1: Wrong condition order

```
-- ✗ General before specific
CASE WHEN score > 0 THEN 'Has Score'
      WHEN score > 100 THEN 'High Score' -- Never reached
END

-- ✓ Specific before general
CASE WHEN score > 100 THEN 'High Score'
      WHEN score > 0 THEN 'Has Score'
END
```

Mistake 2: Forgetting ELSE

```
-- ✗ No ELSE: unmatched rows return NULL
CASE WHEN status = 'open' THEN 'Active'
      WHEN status = 'closed' THEN 'Resolved'
END -- What about 'cold' status? Returns NULL!

-- ✓ Always include ELSE unless you want NULL
CASE WHEN status = 'open' THEN 'Active'
      WHEN status = 'closed' THEN 'Resolved'
      ELSE 'Other'
END
```

Mistake 3: Using = NULL instead of IS NULL

✗ Wrong: Using = NULL

```
CASE WHEN phone = NULL THEN 'No phone'
      ELSE phone
END
```

- = NULL always evaluates to UNKNOWN (not TRUE)
- Every row falls to ELSE
- No rows ever get 'No phone'

✓ Right: Using IS NULL

```
CASE WHEN phone IS NULL THEN 'No phone'
      ELSE phone
END
```

- IS NULL correctly tests for NULL values
- Rows with missing phone numbers now get 'No phone'
- Standard SQL behavior across all databases

Nothing equals NULL in SQL, not even NULL itself. Always use IS NULL.

Quick Reference: CASE WHEN Cheat Sheet

Pattern	Syntax	Use Case
Categorize	CASE WHEN x > 10 THEN 'high' END	Create labels from values
Pivot	COUNT(CASE WHEN status='open' THEN 1 END)	Turn rows into columns
Custom sort	ORDER BY CASE WHEN ... END	Non-alphabetical ordering
NULL handling	CASE WHEN x IS NULL THEN 'N/A' END	Replace missing values

Pattern	Syntax	Use Case
Conditional SUM	SUM(CASE WHEN type='A' THEN amount END)	Sum specific subsets

All CASE WHEN patterns in one place

Test Your CASE WHEN Knowledge

Q1. What happens if no WHEN condition matches and there's no ELSE clause?

An error is thrown

NULL is returned

The first WHEN result is returned

An empty string is returned

Q2. Which CASE form lets you use range comparisons like > or BETWEEN?

Simple CASE

Searched CASE

Both

Neither

Q3. You want to count how many orders are 'high value' (over \$1000). Which pattern works?

COUNT(CASE WHEN amount > 1000 THEN 1 END)

CASE WHEN COUNT(amount) > 1000 THEN 1 END

COUNT(*) WHERE CASE amount > 1000

SUM(CASE WHEN amount > 1000 THEN COUNT(*) END)

NULL is not a valid value in SQL

= NULL always returns UNKNOWN, not TRUE

CASE doesn't support NULL comparisons

You need to use != NULL instead

Ready to put CASE WHEN into practice?

CASE WHEN is one of the most-used SQL patterns in real work and interviews. Build muscle memory by solving detective cases where you classify evidence and crack mysteries with conditional logic.

Start Your Investigation →

FAQ

What is the difference between CASE and IF in SQL?

CASE is standard SQL and works in every database. IF is MySQL-specific (SQL Server has IIF). Use CASE for portability across databases.

Can you use CASE WHEN in a WHERE clause?

Yes, but it's rarely needed. Most CASE-in-WHERE can be rewritten as simple boolean conditions, which are clearer and often faster because the optimizer can use indexes.

Does CASE WHEN affect query performance?

Minimal impact. CASE is evaluated per-row like any other expression. The bigger performance concern is using CASE in WHERE, which can prevent the database from using indexes effectively.

Can you have multiple CASE WHEN in one SELECT?

Yes. Each CASE expression is independent. You can have as many CASE columns as you need in a single SELECT statement.

What is the CASE WHEN pivot pattern?

Using CASE inside aggregate functions (COUNT, SUM) to turn row values into columns. It works in all SQL dialects, unlike the PIVOT keyword which is only available in SQL Server and Oracle.



Related Guides

- [What Is a CTE in SQL?](#) - Organize complex queries with named subqueries
- [HAVING vs WHERE in SQL](#) - Know when to filter rows vs groups
- [SQL Window Functions Explained](#) - Advanced analytics with OVER()
- [SQL for Data Analysts](#) - Essential skills for data analysis careers

Ready to start your next investigation?

We use cookies for analytics. DNT is respected. [Privacy Policy](#)

Jump into the SQLNoir case files and put these tips to work.

Explore Cases 

Back to Journal

Keep investigating with SQL Noir - where mysteries meet databases.

Back to Blog

Play Game